⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Clause: 5.3 — Software Architectural Design · **Register:** Document Register

> Clause 5.3 has **no Class A requirement at all** — every row below applies at Class B and C only, one (5.3.5) at Class C alone.

What the standard requires (Class C — every sub-clause of 5.3)

REF	TITLE	APPLIES AT	OUTPUT
5.3.1	Transform requirements into an architecture	B, C	Documented architecture describing the software structure and identifying the software items
5.3.2	Architecture for interfaces of software items	B, C	Documented architecture for interfaces between software items and external components
5.3.3	Specify functional/performance requirements of SOUP	B, C	Functional and performance requirements specified for each SOUP item
5.3.4	Specify system hardware/software required by SOUP	B, C	System hardware and software specified as necessary to support each SOUP item
5.3.5	Identify segregation necessary for risk control	**C only**	Segregation between software items necessary for risk control, and how its effectiveness is ensured
5.3.6	Verify software architecture	B, C	Documented verification that the architecture implements requirements, supports interfaces, and supports SOUP operation

5.3.1 — Software items

There is no build step and no bundler — every script *is* the software item it defines, one file each:

SOFTWARE ITEM	RESPONSIBILITY
`nav.js`	Shared navigation — highlights the active page link
`async-utils.js`	Shared async helpers: `delay()`, `fetchJSON()` (centralises the `response.ok` check `fetch` doesn't do itself)
`theme.js`	Light/dark theme toggle and persistence

SOFTWARE ITEM	RESPONSIBILITY
`learn.js`	Learn page: card rendering, level toggle, safety-class filter, deliverables list, progress tracker
`quiz.js`	Quiz page: question loading with prefetch, shuffle, timer, scoring, certificate
`contact.js`	Contact page: field validation, asynchronous submission with timeout
`style.css`	All visual presentation, including both theme palettes

Each page loads `nav.js`, then `async-utils.js`, then its own page script, all marked `defer` — deferred scripts run in document order, which is what guarantees the shared helpers exist before a page script calls them (the project's design notes record why this ordering was chosen over a bundler).

5.3.2 — Interfaces between software items

There is no shared global state between pages. Where two items do need to communicate, the interface is one of:

INTERFACE	BETWEEN	MECHANISM
Data loading	`learn.js` / `quiz.js` ↔ `data/*.json`	`fetch()` via `fetchJSON()`, validated for shape on the way in
Cross-page state	`learn.js` → `quiz.js`	`localStorage['62304_trainingLevel']` — the two pages share no JavaScript and communicate only through this key
Theme state	`theme.js` ↔ every page	`localStorage['62304_theme']` plus the `data-theme` attribute on `<html>`
Banner dismissal	`learn.js` (write) / `learn.js` (read on reload)	`localStorage['62304_bannerDismissed']`
Form submission	`contact.js` → Formspree	`fetch()` POST to `https://formspree.io/f/mpqgydrv`, JSON request/response

Every `localStorage` key is namespaced `62304_...` specifically so its owner is unambiguous in DevTools — itself a small interface-design decision worth recording, since an unprefixed key is a common source of collision when a page later gains a second script that also wants "the" storage key.

5.3.3–5.3.4 — SOUP

Nothing third-party ships to a visitor's browser. The production pages load no external script, font, or stylesheet — confirmed by grep across all five HTML files, whose only external references are outbound links (to `stjohnlynch.com`, `askriskie.com`, `iec.ch`, `dataprotection.ie`, and the Formspree endpoint itself) rather than resources the page depends on to render. That is a deliberate architectural property, not an oversight, and it is why the SOUP table below is short:

SOUP ITEM	MANUFACTURER	VERSION	SHIPS TO VISITORS?	FUNCTIONAL/PERFORMANCE REQUIREMENT	ENVIRON REQUIRE
Playwright	Microsoft	`>=1.40` (see `tests/requirements.txt`)	**No — test-only**	Drives a real Chromium/Chrome to exercise the site as a user would	Python 3, Chromiur Chrome i
axe-core	Deque Systems	`4.10.2`, pinned by URL in `tests/test_site.py` (`AXE_CDN`)	**No — test-only**	WCAG 2.1 A/AA + best-practice accessibility scanning	Downloac test time; is skippec failed) if unreacha

Formspree is not SOUP in the 5.3 sense — it is not a software component embedded in the product, it is an external HTTP service the contact form calls at runtime, closer to an *interfacing system* than to third-party code under this project's control. It is documented here for completeness because a reader learning what SOUP means benefits from seeing a real example of something that is deliberately excluded and why.

5.3.5 — Segregation for risk control (Class C only)

The one place this project asks "could a mistake here affect something that matters" is the safety-class data. `learn.js`'s `mergeApplicability()` segregates *rendering* from *trusting*: the merge step is a hard gate — if `phases.json` and `applicability.json` disagree, no card renders at all, rather than rendering with unverified data. See 11 — Risk Management File for the incident that made this segregation necessary.

5.3.6 — Architecture verification

There is no standalone architecture review record — the closest equivalent is that every interface listed above is exercised directly by `tests/test_site.py` (for example, the `learn` and `applicability` groups assert on the `localStorage` keys and the render-gate behaviour described above), which verifies the architecture indirectly, by verifying its observable effects, rather than by a separate design review.

Gaps

- No dedicated architecture diagram exists outside the tables above — for a project this size (seven files) a diagram would likely repeat the file table rather than add information; that trade-off would flip for a larger codebase.
- 5.3.6's verification is behavioural, not a formal design review with reviewer sign-off — recorded as what it is rather than dressed up as one.